```
!pip install nltk
!pip install pandas
!pip install numpy
!pip install scikit-learn
!pip install gensim
!pip install tensorflow

# ============================
# Core Libraries
# ============================
import nltk
import pandas as pd
import numpy as np
import json
import math
import re
import string

# ============================
# Scikit-learn
# ============================
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

# ============================
# Collections
# ============================
from collections import defaultdict, Counter

# ============================
# NLTK - NLP Processing
# ============================
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag, ne_chunk, CFG, PCFG
from nltk.tree import Tree
from nltk.parse import RecursiveDescentParser, ViterbiParser, InsideChartParser
from nltk.collocations import BigramAssocMeasures, BigramCollocationFinder
from nltk.util import bigrams

# ============================
# Gensim
# ============================
from gensim.models import Word2Vec

# ============================
# TensorFlow / Keras
# ============================
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer

# ============================
# NLTK Downloads
# ============================
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('wordnet')
# Download the required resource to prevent the LookupError
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

Show hidden output

EX10 LSTM

EX01

Exp 1: Text Preprocessing Pipeline

Aim: To apply the basic NLP steps (tokenization, stopword removal, stemming, lemmatization, POS tagging, NER) on a text.

Algorithm:

1. Convert the text to lowercase and replace extra whitespace with a single space using re.sub.
2. Sentence tokenization: split the text into sentences with sent_tokenize.
3. Word tokenization: split the text into words with word_tokenize.
4. Stopword removal: keep only the words that are not in the stopword list and are alphanumeric.
5. Stemming: cut each word to its root with the Porter stemmer (sleeping → sleep).
6. Lemmatization: convert each word to its dictionary form with the WordNet lemmatizer (dogs → dog).
7. POS tagging: label each word with its part of speech (NN, VB, JJ...) using pos_tag.
8. Named entity recognition: group the tagged words into entities (person, place, organization) with ne_chunk.

```
import re
raw = "the quick brown fox jumps over the lazy dog. the lazy dog decided to sleep all day under the big oak tree. meanwhile the brown fox w

# Fix: Clean up newlines and extra spaces, but keep the words and punctuation intact
sentence = re.sub(r"\s+", " ", raw.lower())

#---
sentence_tokens = sent_tokenize(sentence)
word_tokens = word_tokenize(sentence)
#---
stop_words = set(stopwords.words('english'))
words = [w for w in word_tokens if w not in stop_words and w.isalnum()]
#---
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(w) for w in words]
#---
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(w) for w in words]
#---
tagged_words = pos_tag(words)
#---
nammed_entities = ne_chunk(tagged_words)
#---
grammer = CFG.fromstring("""
S   -> NP VP
NP  -> Det N
VP  -> V NP
Det -> 'the' | 'a'
N   -> 'boy' | 'dog'
V   -> 'saw'
""")

parser = nltk.ChartParser(grammer)
test_words = ['the', 'boy', 'saw', 'a', 'dog']
trees = parser.parse(test_words)
for tree in trees:
  print(tree)
  tree.pretty_print()
```

EX02

Exp 2: Collocation Extraction (t-test and Chi-square)

Aim: To find word pairs that occur together more often than by chance, using the t-test and the chi-square test.

Algorithm:

1. Take the text and remove stopwords to get the word list. Let N = number of words.
2. Count the frequency of every word.
3. Form bigrams (adjacent word pairs) and count each bigram's frequency (observed).
4. Compute the expected count under independence: f(w1) × f(w2) ÷ N.
5. t-test: t = (observed − expected) ÷ √observed. If t > 1.645, the pair is a collocation.
6. Chi-square: build a 2×2 table (w1w2, w1 with another word, another word with w2, neither).
7. Compute each cell's expected count = row total × column total ÷ grand total, then $\chi^2 = \Sigma\ (O - E)^2 \div E$.
8. If $\chi^2 > 2.706$ (critical value), reject H0 (the words are independent) and call the pair a collocation. Print the result for each bigram.

```
import math
from collections import Counter

# your text after stopword removal (replace with your own words list)
words = "knocked door knocked door man knocked metal front door knocked door".split()

N = len(words)
word_freq = Counter(words)
bigrams = list(zip(words, words[1:]))        # keeps repeats, so every occurrence is printed
bigram_freq = Counter(bigrams)
total = len(bigrams)
```

```
# ---------- t-test ----------
print("t-test:\n")
for w1, w2 in bigrams:
    obs = bigram_freq[(w1, w2)]
    expected = word_freq[w1] * word_freq[w2] / N
    t = (obs - expected) / math.sqrt(obs)
    result = "are collocated!" if t > 1.645 else "are not collocated!"   # 1.645 = 95% one-tailed
    print(f"t-value: {round(t, 2)}\tWords: {w1} and {w2} {result}")

# ---------- chi-square test ----------
print("\nchi^2-test:\n")
for w1, w2 in bigrams:
    obs = bigram_freq[(w1, w2)]
    f1, f2 = word_freq[w1], word_freq[w2]

    # 2x2 table: [w1 w2 | w1 other] / [other w2 | other other]
    O = [[obs,      f1 - obs],
         [f2 - obs, total - f1 - f2 + obs]]
    chi = 0
    for i in range(2):
        for j in range(2):
            E = sum(O[i]) * (O[0][j] + O[1][j]) / total
            chi += (O[i][j] - E) ** 2 / E

    result = "are collocated!" if chi > 2.706 else "are not collocated!"   # 2.706 = 90%, 1 d.f.
    print(f"chi-value: {chi}\tWords: {w1} and {w2} {result}")
```

Show hidden output

Ex03

Word Sense Disambiguation (Naive Bayes)

Aim: To find the sense of the word “chair” (furniture or position) using a Naive Bayes classifier.

Algorithm:

1. Collect training sentences for each sense (furniture, position).
2. Split each sense’s sentences into words and count the frequency of every word.
3. Build the vocabulary (V) from the unique words of both senses.
4. Compute the prior of each sense: log(sentences in sense ÷ total sentences).
5. Take the test sentence, lowercase it and split it into words.
6. For each word (skip “chair”, since it is in both senses), compute log((count in sense + 1) ÷ (total words in sense + V)).
7. The +1 is Laplace smoothing, so unseen words don’t give zero probability.
8. Add these values to each sense’s score.
9. Choose the sense with the higher score as the meaning of “chair”.

```
import math
from collections import Counter

furniture = ["chair table sofa furniture", "chair sit wooden furniture",
             "chair desk room furniture", "chair comfortable seat", "wooden chair table"]
position = ["chair meeting committee leader", "chair president meeting",
            "chair department head", "chair committee discussion", "chair leader organization"]
sentence = "The chair attended the meeting with the committee"

# word counts per class (keeps repeats)
F = Counter(" ".join(furniture).split())
P = Counter(" ".join(position).split())
V = len(set(F) | set(P))                  # vocabulary = unique WORDS
F_total, P_total = sum(F.values()), sum(P.values())   # total tokens per class

# priors from sentence counts
score_f = math.log(len(furniture) / (len(furniture) + len(position)))
score_p = math.log(len(position) / (len(furniture) + len(position)))

for w in sentence.lower().split():
    if w == "chair":
        continue                          # the target word is in both classes
    score_f += math.log((F[w] + 1) / (F_total + V))   # +1 = Laplace smoothing
    score_p += math.log((P[w] + 1) / (P_total + V))

print("Furniture Score:", score_f)
print("Position Score:", score_p)
print("The sense of 'chair' is", "furniture" if score_f > score_p else "position")
```

```
Furniture Score: -30.375055467112254
Position Score: -30.231581787659554
The sense of ‘Chair’ is position: -30.231581787659554
```

Ex04

Exp 4: Hindle and Rooth (PP Attachment)

Aim: To decide whether a prepositional phrase (PP) attaches to the verb or to the noun.

Algorithm:

1. Collect a large corpus of sentences containing prepositional phrases.
2. Parse or tag the sentences to find the verb, noun and preposition in each pattern (verb + noun + preposition).
3. Count C(v) and C(n): how often each verb and noun occurs.
4. Count C(v,p) and C(n,p): how often the preposition attaches to each verb and noun.
5. Compute P(VA=1|v) = C(v,p) ÷ C(v) and P(NA=1|n) = C(n,p) ÷ C(n). Add smoothing so unseen words don't cause division by zero.
6. For a new sentence, find its verb, noun and preposition.
7. Compute $\lambda = \log_2 [P(VA=1|v) \times (1 - P(NA=1|n)) \div P(NA=1|n)]$.
8. If $\lambda > 0$, attach the PP to the verb. Otherwise attach it to the noun.

```
import math
from collections import Counter

import nltk
from nltk import pos_tag, sent_tokenize, word_tokenize

# Replace with a large corpus (e.g. nltk.corpus.brown.raw()) for real use
text = """
He ate the pizza with a fork. She saw the man in the park.
They depend on the results. The report on sales was published.
He put the book on the table. The house on the hill is old.
She relied on the manager. The decline in profits worried investors.
He opened the door with a key. We discussed the problem with the team.
"""

sents = [pos_tag(word_tokenize(s.lower())) for s in sent_tokenize(text)]

V_count, N_count = Counter(), Counter()
Vp_count, Np_count = Counter(), Counter()
triples = []

# Pass 1: C(v) and C(n)
for sent in sents:
    for w, tag in sent:
        if tag.startswith("VB"):
            V_count[w] += 1
        elif tag.startswith("NN"):
            N_count[w] += 1

# Pass 2: C(v,p) and C(n,p) from unambiguous cases; ambiguous ones become triples
for sent in sents:
    for i, (w, tag) in enumerate(sent):
        if tag != "IN":
            continue
        before = sent[:i]
        v = next((x for x, t in reversed(before) if t.startswith("VB")), None)
        n = next((x for x, t in reversed(before) if t.startswith("NN")), None)
        if before and before[-1][1].startswith("VB"):
            Vp_count[(before[-1][0], w)] += 1      # verb attachment
        elif n and not v:
            Np_count[(n, w)] += 1                  # noun attachment
        elif v and n:
            triples.append((v, n, w))              # ambiguous

# Score ambiguous triples
alpha = 0.5
print(f"{'verb':<10}{'noun':<10}{'prep':<8}{'lambda':>8}  attach")
for v, n, p in triples:
    pv = (Vp_count[(v, p)] + alpha) / (V_count[v] + 2 * alpha)
    pn = (Np_count[(n, p)] + alpha) / (N_count[n] + 2 * alpha)
    lam = math.log2(pv * (1 - pn) / pn)
    print(f"{v:<10}{n:<10}{p:<8}{lam:>8.2f}  {'verb' if lam > 0 else 'noun'}")

# New sentence
v, n, p = "ate", "pizza", "with"
pv = (Vp_count[(v, p)] + alpha) / (V_count[v] + 2 * alpha)
pn = (Np_count[(n, p)] + alpha) / (N_count[n] + 2 * alpha)
lam = math.log2(pv * (1 - pn) / pn)
print(f"\n'{v} {n} {p} ...' -> lambda = {lam:.2f}, attach to {'verb' if lam > 0 else 'noun'}")
```

```
  File "/tmp/ipykernel_1693/2987601795.py", line 8
    if w,tag in sent:
       ^
SyntaxError: invalid syntax
```

Next steps: Explain error

Ex05

Exp 5: Forward and Backward Algorithms (Hidden Markov Model)

Aim: To compute the probability of an observation sequence under a Hidden Markov Model (HMM) using the forward and backward algorithms.

Algorithm:

1. Define the HMM: the hidden states (Sunny, Rainy), the initial probabilities π, the transition probabilities A and the emission probabilities B.
2. Take the observation sequence O = (o_1, o_2, ..., o_T), for example Walk, Shop, Walk.

Forward algorithm

3. Initialization: for each state i, compute $\alpha_1(i) = \pi(i) \times B(i, o_1)$.
4. Recursion: for t = 2 to T and each state j, compute $\alpha_t(j) = [\ \Sigma_i\ \alpha_{t-1}(i) \times A(i, j)\ ] \times B(j, o_t)$.
5. Termination: $P(O \mid model) = \Sigma_i\ \alpha_T(i)$.

Backward algorithm

6. Initialization: for each state i, set β_T(i) = 1.
7. Recursion: for t = T−1 down to 1 and each state i, compute $\beta_t(i) = \Sigma_j\ A(i, j) \times B(j, o_{t+1}) \times \beta_{t+1}(j)$.
8. Termination: $P(O \mid model) = \Sigma_i\ \pi(i) \times B(i, o_1) \times \beta_1(i)$.

Result

9. The forward and backward probabilities must be equal, which confirms both computations are correct.

```
# Number of states
states = ['Sunny', 'Rainy']
n = len(states)

# Initial probabilities
initial = [0.6, 0.4]

# Transition probabilities
transition = {
    ('Sunny', 'Sunny'): 0.7,
    ('Sunny', 'Rainy'): 0.3,
    ('Rainy', 'Sunny'): 0.4,
    ('Rainy', 'Rainy'): 0.6
}

# Emission probabilities
emissions = {
    'Sunny': {
        'Walk': 0.8,
        'Shop': 0.2
    },
    'Rainy': {
        'Walk': 0.3,
        'Shop': 0.7
    }
}

# Observation sequence
observations = ['Walk', 'Shop', 'Walk']
T = len(observations)

# --------------------------------
# FORWARD ALGORITHM
# --------------------------------

forward = []

# Initialization
first = []

for state in states:
    probability = (
        initial[states.index(state)]
```

```
            * emissions[state].get(observations[0], 0)
        )

        first.append(probability)

    forward.append(first)


    # Recursion
    for t in range(1, T):

        current = []

        for j in range(n):

            total = 0

            for i in range(n):
                total += forward[t - 1][i]* transition[(states[i], states[j])]


            total *= emissions[states[j]].get(observations[t], 0)

            current.append(total)

        forward.append(current)


    # Termination
    forward_probability = sum(forward[T - 1])


    print("FORWARD PROBABILITY:", forward_probability)

    # ---------------------------------
    # BACKWARD ALGORITHM
    # ---------------------------------

    backward = [[0]*n for _ in range(T)]
    backward[T-1] = [1] * n                      # initialization: last step is 1

    for t in range(T-2, -1, -1):
        backward[t] = [sum(transition[(states[i], states[j])]*emissions[states[j]][observations[t+1]]* backward[t+1][j]for j in range(n))for

    backward_probability = sum(
        initial[i] * emissions[states[i]][observations[0]] * backward[0][i]
        for i in range(n)
    )
    print("BACKWARD PROBABILITY:", backward_probability)
```

Ex06

Exp 6: Viterbi Algorithm (Hidden Markov Model)

Aim: To find the most probable sequence of hidden states for a given observation sequence using the Viterbi algorithm.

Algorithm:

1. Define the HMM: the hidden states, the initial probabilities π, the transition probabilities A and the emission probabilities B.
2. Take the observation sequence O = (o_1, o_2, ..., o_T).
3. Initialization: for each state i, compute $v_1(i) = \pi(i) \times B(i, o_1)$.
4. Recursion: for t = 2 to T and each state j, compute $v_t(j) = \max_i [\, v_{t-1}(i) \times A(i, j) \,] \times B(j, o_t)$.
5. Backpointer: for each $v_t(j)$, store the previous state i that gave the maximum, $b_t(j) = \text{argmax}_i [\, v_{t-1}(i) \times A(i, j) \,]$.
6. Termination: the best path probability is $\max_i$ v_T(i), and the last state of the best path is argmax_i v_T(i).
7. Backtracking: starting from the last state, follow the backpointers from t = T down to 2 to recover the earlier states.
8. Reverse the collected states to get the best path q_1, q_2, ..., q_T, and output it with its probability.

```
def viterbi(observations,emission,transition,states,initial):
  v = []
  path = []
  first = {}

  for state in states:
    first[state] = initial[state]*emission[state][observations[0]]

  v.append(first)

  for t in range(1,len(observations)):
    current = {}
    current_path = {}
```

```
    for state in states:
      maxp = 0
      best_prev = None

      for prev in states:
        p = transition[prev][state]*v[t-1][prev]*emission[state][observation[t]]

        if p>maxp:
          maxp = p
          best_prev = prev

      current[state] = maxp
      current_path[state] = best_prev

    v.append(current)
    path.append(cuurent_path)


  last_state = max(v[-1],key=v[-1].get)
  best_prob = v[-1][last_state]

  best_path = [last_state]

  for i in range(len(path)-1,-1,-1):
    last_state = path[i][last_state]
    best_path.append(last_state)

  best_path.reverse()
  return best_prob,best_path
```

Ex07

Exp 7: PCFG using the CYK Algorithm (Inside Probability)

Aim: To compute the probability of a sentence under a Probabilistic Context-Free Grammar (PCFG) using the CYK algorithm and inside probabilities.

Algorithm:

1. Define the PCFG in Chomsky Normal Form: each rule is either A → B C (two non-terminals) or A → w (a word), with a probability attached.
2. Take the sentence $w_1, w_2, \ldots, w_n$ and create an n × n table, where table[i][j] stores the inside probability of each symbol A spanning words i to j.
3. Base case: for each word w_i and each rule A → w_i with probability p, set table[i][i][A] = p.
4. Recursion: for span lengths 2 to n, for each start i (with end j = i + length − 1), and for each split point k from i to j−1:
   - For each rule A → B C with probability p, if B is in table[i][k] and C is in table[k+1][j], compute table[i][k][B] × table[k+1][j][C] × p.
   - Add this value to table[i][j][A], so that all the different ways of building A over the span are summed.
5. Termination: the probability of the sentence is the inside probability of the start symbol, P(sentence) = table[0][n−1][S].
6. Display the inside probabilities of all symbols over all spans.

```
"""Ex 7: PCFG using CYK Algorithm — inside probability."""

# ============================================================
# GRAMMAR RULES WITH PROBABILITIES
# ============================================================

grammar = {
    ("S",   ("NP", "VP")): 1.0,
    ("NP",  ("Det", "N")): 0.8,
    ("VP",  ("V",  "NP")): 0.7,
    ("Det", ("the",)):     1.0,
    ("N",   ("cat",)):     0.5,
    ("N",   ("dog",)):     0.5,
    ("V",   ("chased",)):  1.0,
}

sentence = "the cat chased the dog".split()
n        = len(sentence)

# ============================================================
# CYK TABLE INITIALIZATION
# ============================================================

# table[i][j][symbol] = inside probability of 'symbol' spanning words i..j
```

```
table = [[{} for _ in range(n)] for _ in range(n)]

# ============================================================
# BASE CASE: SINGLE WORDS (length = 1)
# ============================================================

for i in range(n):
    word = sentence[i]
    for (lhs, rhs), prob in grammar.items():
        if rhs == (word,):
            table[i][i][lhs] = prob

# ============================================================
# RECURSIVE CASE: LONGER SPANS
# ============================================================

for length in range(2, n + 1):              # span length
    for i in range(n - length + 1):         # start index
        j = i + length - 1                  # end index
        for k in range(i, j):               # split point
            for (lhs, rhs), prob in grammar.items():
                if len(rhs) == 2:
                    B, C = rhs
                    if B in table[i][k] and C in table[k+1][j]:
                        value = table[i][k][B] * table[k+1][j][C] * prob
                        table[i][j][lhs] = table[i][j].get(lhs, 0) + value

# ============================================================
# DISPLAY RESULTS
# ============================================================

print("Inside Probabilities:")
for i in range(n):
    for j in range(i, n):
        for symbol, probability in table[i][j].items():
            print(f"  {symbol} from '{sentence[i]}' to '{sentence[j]}' = {round(probability, 6)}")

print("\nFinal Probabilities (full sentence span):")
print("P(sentence) = inside(S) =", table[0][n-1].get("S",   0))
print("Inside probability of NP:", table[0][n-1].get("NP",  0))
print("Inside probability of VP:", table[0][n-1].get("VP",  0))
print("Inside probability of Det:", table[0][n-1].get("Det", 0))
print("Inside probability of N:",  table[0][n-1].get("N",   0))
```

Ex08

Exp 8: Text Classification using Deep Learning (BoW / TF-IDF)

Aim: To classify text documents into categories (Sports, Technology, Politics) using a neural network trained on Bag of Words or TF-IDF features.

Algorithm:

1. Collect a set of text documents with a category label for each.
2. Convert the text labels into numbers using a label encoder (for example Politics = 0, Sports = 1, Technology = 2).
3. Choose the feature method:
    - Bag of Words (BoW): each document becomes a vector of word counts.
    - TF-IDF: each document becomes a vector of word counts weighted by how rare the word is across all documents. Remove common stop words before building the vectors.
4. Split the data into a training set and a test set.
5. Build the neural network:
    - Input layer: one input for each word in the vocabulary.
    - Hidden layer: 16 neurons with ReLU activation.
    - Output layer: 3 neurons (one per category) with softmax activation.
6. Compile the model with the Adam optimizer and sparse categorical cross-entropy loss, and track accuracy.
7. Train the model on the training set for 30 epochs.
8. Predict on the test set. For each document, the predicted class is the output neuron with the highest probability (argmax).
9. Evaluate the predictions with a classification report (precision, recall, F1-score and accuracy).

```
import tensorflow as tf
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
```

```
documents = [
    "The team won the cricket match by scoring many runs",
    "The player scored a century in the cricket match",
    "The football team won the championship",
    "The goalkeeper saved the final goal",
    "The batsman played an excellent innings",
    "The new smartphone has a powerful processor",
    "Artificial intelligence is changing modern technology",
    "The computer uses machine learning algorithms",
    "The new laptop has a fast processor and large memory",
    "Machine learning is widely used in artificial intelligence",
    "The government announced new economic policies",
    "The election campaign started this week",
    "The minister discussed new government policies",
    "The political party won the election",
    "The parliament passed a new law"
]

labels = [
    "Sports", "Sports", "Sports", "Sports", "Sports",
    "Technology", "Technology", "Technology", "Technology",
    "Technology", "Politics", "Politics", "Politics",
    "Politics", "Politics"
]

encoder=LabelEncoder()
y=encoder.fit_transform(labels)

choice=int(input("Enter the choice (1.tfidf) (2.bow)"))
if choice==2:
  vector=CountVectorizer(stop_words='english')
else:
  vector=TfidfVectorizer(stop_words='english')

x=vector.fit_transform(documents).toarray()
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2)

#trainning deeplearing model
model= tf.keras.Sequential([
    tf.keras.layers.Input((x_train.shape[1],)),
    tf.keras.layers.Dense(16,activation='relu'),
    tf.keras.layers.Dense(3,activation='softmax')

])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # <--- Add this line
    metrics=['accuracy']
)

model.fit(
    x_train,y_train,
    epochs=30
)


prediction = model.predict(x_test)

import numpy as np
print(prediction)
y_pred=np.argmax(prediction,axis=1)
print(classification_report(y_test,y_pred))
```

Ex09

Exp 9: Word Embeddings using Word2Vec (CBOW and Skip-gram)

Aim: To learn word vectors from text using the CBOW and Skip-gram models of Word2Vec, and to find the similarity between words.

Algorithm:

1. Collect the text corpus and convert it to lowercase.
2. Split the text into sentences, then split each sentence into word tokens.
3. Build the CBOW model (sg = 0): it learns word vectors by predicting a target word from its surrounding context words.
4. Build the Skip-gram model (sg = 1): it learns word vectors by predicting the surrounding context words from a target word.
5. Train both models on the tokenized sentences, keeping every word (min_count = 1) because the corpus is small.
6. Find the similarity between two words as the cosine similarity of their word vectors.
7. Find the most similar words to a given word by ranking all other words by their cosine similarity to it.
8. Compare the results of the CBOW and Skip-gram models.

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import re

text = "Alice was beginning to get very tired of sitting by her sister on the bank and of having nothing to do. Alice went down the rabbi

sentences=[]
for sentence in nltk.sent_tokenize(text.lower()):
  words=word_tokenize(sentence)
  if words:
    sentences.append(words)

print("Tokenized Sentences:")
print(sentences)
cbow_model = Word2Vec(sentences=sentences,min_count=1,sg=0)
skipgram_model = Word2Vec(sentences=sentences,min_count=1,sg=1)

print("Similarity of Alice and Machine:",cbow_model.wv.similarity('alice','machine'))

print("Similarity of Alice and Machine:",skipgram_model.wv.similarity('alice','machine'))

print("Most Similar of Alice:",cbow_model.wv.most_similar("alice",topn = 3))
```

Ex10

Exp 10: Text Generation using LSTM

Aim: To design a text generator using LSTM that predicts and generates words based on the input given by the user from the available corpus.

Algorithm:

1. Load the necessary libraries (numpy, tensorflow, keras, re) and read the text corpus.
2. Clean the text: convert it to lowercase and remove punctuation, then split it into lines (sentences).
3. Build the vocabulary: collect all unique words, reserve index 0 for padding, and create the dictionaries word → index and index → word.
4. Generate n-gram sequences: for each sentence, create every prefix of length 2, 3, 4, ... so each prefix becomes one training sequence.
5. Pad all sequences with zeros at the front so they have the same length (max_sequence_len).
6. Split each sequence into predictors X (all words except the last) and label Y (the last word).
7. Build the model:
    - Embedding layer: converts each word index into a dense vector.
    - LSTM layer: learns the order of the words using its forget, input and output gates.
    - Dense layer with softmax: gives the probability of each word in the vocabulary being the next word.
8. Compile the model with the Adam optimizer and sparse categorical cross-entropy loss, then train it on X and Y.
9. Generate text:
    - Convert the seed words to indices and pad them to the input length.
    - Predict the next word as the one with the highest probability (argmax).
    - Append that word to the seed and repeat for the required number of words.
10. Display the generated text.

```
import os
import re
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

# ---------- 1. LOAD DATA ----------
# read corpus.txt if present, else use a tiny sample text
if os.path.exists("corpus.txt"):
    text = open("corpus.txt", encoding="utf-8").read().lower()
else:
    text = """the united states talks about new policy
the united states signs a new trade deal
the president talks about the new policy
the president signs a new law
science and technology are changing the world
science and technology help the people"""

# ---------- 2. CLEAN DATA ----------
text = re.sub(r"[^a-z\s]", "", text)                 # keep only letters and spaces (removes punctuation)
lines = [l for l in text.split("\n") if l.strip()]    # one line = one sentence, drop empty lines

# ---------- 3. VOCABULARY (word <-> number) ----------
vocab = ["<pad>"] + sorted(set(text.split()))         # index 0 is reserved for padding
```

```
w2i = {w: i for i, w in enumerate(vocab)}            # word -> index
i2w = {i: w for w, i in w2i.items()}                 # index -> word
V = len(vocab)                                       # vocabulary size

# ---------- 4. N-GRAM SEQUENCES ----------
# "the united states talks" -> [the united] [the united states] [the united states talks]
sequences = []
for line in lines:
    ids = [w2i[w] for w in line.split()]
    for i in range(1, len(ids)):
        sequences.append(ids[:i + 1])                # every prefix of length 2, 3, 4, ...

max_sequence_len = max(len(s) for s in sequences)
sequences = tf.keras.utils.pad_sequences(sequences, maxlen=max_sequence_len, padding="pre")  # pad zeros at the front

X = sequences[:, :-1]                                # predictors = all words except the last
Y = sequences[:, -1]                                 # label = the last word (the one to predict)

# ---------- 5. MODEL ----------
model = models.Sequential([
    layers.Input(shape=(max_sequence_len - 1,)),
    layers.Embedding(V, 100),                        # word index -> 100-d vector
    layers.LSTM(150),                                # LSTM remembers the word order
    layers.Dense(V, activation="softmax"),           # probability of each word being next
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(X, Y, epochs=100, verbose=0)               # train: predictors X -> label Y

# ---------- 6. GENERATE TEXT ----------
def generate_text(seed, n_words, model, max_sequence_len):
    out = [w for w in seed.lower().split() if w in w2i]      # keep only known words
    for _ in range(n_words):
        ids = [w2i[w] for w in out]
        x = tf.keras.utils.pad_sequences([ids], maxlen=max_sequence_len - 1)  # pad / trim to model input size
        p = np.argmax(model.predict(x, verbose=0)[0])        # most probable next word
        out.append(i2w[p])                                   # add it and repeat
    return " ".join(out)

print(generate_text("united states", 10, model, max_sequence_len))
print(generate_text("science and technology", 10, model, max_sequence_len))
```

Ex11

Exp 11: Machine Translation using RNN (Encoder–Decoder)

Aim: To design a character-level translator from English to French using an encoder–decoder model built with a Recurrent Neural Network (RNN).

Algorithm:

1. Load the necessary libraries (numpy, tensorflow, keras) and collect the English–French sentence pairs.
2. Add a start symbol (\t) and an end symbol (\n) to every French sentence.
3. Build the character dictionaries for the English and French sides (character → index).
4. Find the maximum sentence length on both sides.
5. Convert the sentences to one-hot arrays:
    - X: the English sentence (encoder input).
    - D: the French sentence (decoder input).
    - Y: the French sentence shifted one step ahead (decoder target).
6. Build the encoder: an RNN reads the English sentence and returns only its final state h, which summarizes the sentence.
7. Build the decoder: an RNN starts from the encoder's state h and reads the French characters, followed by a Dense layer with softmax that predicts the next character.
8. Compile the model with the RMSprop optimizer and categorical cross-entropy loss, then train it on [X, D] → Y.
9. Translate a new sentence:
    - One-hot encode the English sentence.
    - Start the output with the start symbol.
    - Predict the next character, add it to the output and feed it back in.
    - Repeat until the end symbol is predicted or the maximum length is reached.
10. Remove the start symbol and display the translated sentence.

```
import os
import numpy as np
from tensorflow.keras.layers import SimpleRNN, Dense, Input
from tensorflow.keras.models import Model

# ---------- 1. DATA (English -> French) ----------
```

```
pairs = [("Go.", "Va !"), ("Run!", "Cours !"), ("Who?", "Qui ?"), ("Wow!", "Ça alors !"),
         ("Fire!", "Au feu !"), ("Help!", "À l'aide !"), ("Jump.", "Saute."), ("Stop!", "Ça suffit !"),
         ("Wait!", "Attends !"), ("Hello!", "Bonjour !"), ("I try.", "J'essaye."), ("I won!", "J'ai gagné !"),
         ("Oh no!", "Oh non !"), ("Attack!", "Attaque !"), ("Cheers!", "Santé !"), ("Get up.", "Lève-toi."),
         ("Go on.", "Poursuis."), ("I know.", "Je sais."), ("I left.", "Je suis parti."), ("Hug me.", "Serre-moi dans tes bras !")]
if os.path.exists("fra.txt"):                          # use the big file if available
    lines = open("fra.txt", encoding="utf-8").read().split("\n")[:5000]
    pairs = [l.split("\t")[:2] for l in lines if "\t" in l]

src = [e for e, f in pairs]                            # English sentences
tgt = ["\t" + f + "\n" for e, f in pairs]              # French sentences: \t = start, \n = end

# ---------- 2. CHARACTER DICTIONARIES ----------
src_chars = sorted(set("".join(src)))
tgt_chars = sorted(set("".join(tgt)))
s2i = {c: i for i, c in enumerate(src_chars)}          # source char -> index
t2i = {c: i for i, c in enumerate(tgt_chars)}          # target char -> index

max_src, max_tgt = max(map(len, src)), max(map(len, tgt))

# ---------- 3. ONE-HOT ARRAYS ----------
X = np.zeros((len(pairs), max_src, len(src_chars)), dtype="float32")   # encoder input
D = np.zeros((len(pairs), max_tgt, len(tgt_chars)), dtype="float32")   # decoder input
Y = np.zeros_like(D)                                                    # decoder target

for n in range(len(pairs)):
    for i, c in enumerate(src[n]):
        X[n, i, s2i[c]] = 1                            # English char i
    for i, c in enumerate(tgt[n]):
        D[n, i, t2i[c]] = 1                            # French char i (input)
        if i > 0:
            Y[n, i - 1, t2i[c]] = 1                    # target = input shifted one step ahead

# ---------- 4. MODEL (encoder - decoder) ----------
encoder_in = Input(shape=(None, len(src_chars)))
decoder_in = Input(shape=(None, len(tgt_chars)))

_, h = SimpleRNN(256, return_state=True)(encoder_in)                  # encoder: keep only the final state h
decoder_seq = SimpleRNN(256, return_sequences=True)(decoder_in, initial_state=h)  # decoder starts from h
dec_out = Dense(len(tgt_chars), activation="softmax")(decoder_seq)    # next-char probabilities

model = Model([encoder_in, decoder_in], dec_out)
model.compile(optimizer="rmsprop", loss="categorical_crossentropy")
model.fit([X, D], Y, epochs=300, batch_size=4, verbose=0)             # train

# ---------- 5. TRANSLATE ----------
def translate(sentence):
    x = np.zeros((1, max_src, len(src_chars)))
    for i, ch in enumerate(sentence[:max_src]):
        if ch in s2i:
            x[0, i, s2i[ch]] = 1                       # one-hot the English input
    out = "\t"                                         # start with the start symbol

    for _ in range(max_tgt - 1):
        d = np.zeros((1, max_tgt, len(tgt_chars)))
        for i, ch in enumerate(out):
            d[0, i, t2i[ch]] = 1                       # feed the French chars so far
        p = model.predict([x, d], verbose=0)[0, len(out) - 1]   # prediction for the next char
        ch = tgt_chars[np.argmax(p)]
        if ch == "\n":                                 # end symbol -> stop
            break
        out += ch
    return out[1:]                                     # remove the start symbol

print(translate("Go."))
print(translate("Hello!"))
print(translate("Help!"))
```